

TuneWorks_Visualizations

```
import sqlite3
```

```

from sqlalchemy import create_engine
import pandas as pd

engine = create_engine("postgresql://username:password@localhost:5432/your_db")

df = pd.read_sql("SELECT * FROM your_table;", engine)
df.head()

```

OperationalError

(psycopg2.OperationalError) connection to server at "localhost" (::1), port 5432 failed: FATAL: role "username" does not exist

(Background on this error at: <https://sqlalche.me/e/20/e3q8>)

```

[31m----- [39m
[31mOperationalError [39m                                Traceback (most recent call last)
[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/base.py:143 [39m, in [36mConnection.__init__ [39m [34m(self, engine, connection,
_has_events, _allow_revalidate, _allow_autobegin) [39m
[32m    142 [39m [38;5;28;01mtry [39;00m:
[32m--> [39m [32m143 [39m [38;5;28;01mself [39m._dbapi_connection =
[43mengine [49m [43m. [49m [43mraw_connection [49m [43m( [49m [43m) [49m
[32m    144 [39m [38;5;28;01mexcept [39;00m dialect.loaded_dbapi.Error [38;5;28;01mas [39;00m err:

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/base.py:3301 [39m, in [36mEngine.raw_connection [39m [34m(self) [39m
[32m    3280 [39m [38;5;250m [39m [33;03m""Return a "raw" DBAPI connection from the connection pool. [39;00m
[32m    3281 [39m
[32m    3282 [39m [33;03mThe returned object is a proxied version of the DBAPI [39;00m
[32m    (...) [39m [32m    3299 [39m
[32m    3300 [39m [33;03m"" [39;00m
[32m--> [39m [32m3301 [39m [38;5;28;01mreturn [39;00m
[38;5;28;43mself [39;49m [43m. [49m [43mpool [49m [43m. [49m [43mconnect [49m [43m( [49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:447 [39m, in [36mPool.connect [39m [34m(self) [39m
[32m    440 [39m [38;5;250m [39m [33;03m""Return a DBAPI connection from the pool. [39;00m
[32m    441 [39m
[32m    442 [39m [33;03mThe connection is instrumented such that when its [39;00m
[32m    (...) [39m [32m    445 [39m
[32m    446 [39m [33;03m"" [39;00m
[32m--> [39m [32m447 [39m [38;5;28;01mreturn [39;00m
[43m_ConnectionFairy [49m [43m. [49m [43m_checkout [49m [43m( [49m [38;5;28;43mself [39;49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:1264 [39m, in [36m_ConnectionFairy._checkout [39m [34m(cls, pool, threadconns,
fairy) [39m
[32m    1263 [39m [38;5;28;01mif [39;00m [38;5;129;01mnot [39;00m fairy:
[32m--> [39m [32m1264 [39m    fairy =
[43m_ConnectionRecord [49m [43m. [49m [43m_checkout [49m [43m( [49m [43mpool [49m [43m) [49m
[32m    1266 [39m [38;5;28;01mif [39;00m threadconns [38;5;129;01mis [39;00m [38;5;129;01mnot [39;00m
[38;5;28;01mNone [39;00m:

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:711 [39m, in [36m_ConnectionRecord._checkout [39m [34m(cls, pool) [39m
[32m    710 [39m [38;5;28;01melse [39;00m:
[32m--> [39m [32m711 [39m    rec = [43mpool [49m [43m. [49m [43m_do_get [49m [43m( [49m [43m) [49m
[32m    713 [39m [38;5;28;01mtry [39;00m:

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/impl.py:177 [39m, in [36mQueuePool._do_get [39m [34m(self) [39m
[32m    176 [39m [38;5;28;01mexcept [39;00m:
[32m--> [39m [32m177 [39m [43m [49m [38;5;28;43;01mwith [39;49;00m [43m
[49m [43mutil [49m [43m. [49m [43msafe_reraise [49m [43m( [49m [43m) [49m [43m: [49m

```

```

[32m 178 [39m [43m [49m [38;5;28;43mself [39;49m [43m. [49m [43m_dec_overflow [49m [43m( [49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/util/langhelpers.py:224 [39m, in [36msafe_reraise.__exit__ [39m [34m(self, type_, value,
traceback) [39m
[32m 223 [39m [38;5;28mself [39m._exc_info = [38;5;28;01mNone [39;00m [38;5;66;03m# remove potential
circular references [39;00m
[32m--> [39m [32m224 [39m [38;5;28;01mraise [39;00m exc_value.with_traceback(exc_tb)
[32m 225 [39m [38;5;28;01melse [39;00m:

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/impl.py:175 [39m, in [36mQueuePool._do_get [39m [34m(self) [39m
[32m 174 [39m [38;5;28;01mtry [39;00m:
[32m--> [39m [32m175 [39m [38;5;28;01mreturn [39;00m
[38;5;28;43mself [39;49m [43m. [49m [43m_create_connection [49m [43m( [49m [43m) [49m
[32m 176 [39m [38;5;28;01mexcept [39;00m:

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:388 [39m, in [36mPool._create_connection [39m [34m(self) [39m
[32m 386 [39m [38;5;250m [39m [33;03m""""Called by subclasses to create a new ConnectionRecord.""" [39;00m
[32m--> [39m [32m388 [39m [38;5;28;01mreturn [39;00m
[43m_ConnectionRecord [49m [43m( [49m [38;5;28;43mself [39;49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:673 [39m, in [36m_ConnectionRecord.__init__ [39m [34m(self, pool, connect) [39m
[32m 672 [39m [38;5;28;01mif [39;00m connect:
[32m--> [39m [32m673 [39m [38;5;28;43mself [39;49m [43m. [49m [43m__connect [49m [43m( [49m [43m) [49m
[32m 674 [39m [38;5;28mself [39m.finalize_callback = deque()

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:899 [39m, in [36m_ConnectionRecord.__connect [39m [34m(self) [39m
[32m 898 [39m [38;5;28;01mexcept [39;00m [38;5;167;01mBaseException [39;00m [38;5;28;01mas [39;00m e:
[32m--> [39m [32m899 [39m [43m [49m [38;5;28;43;01mwith [39;49;00m [43m
[49m [43mutil [49m [43m. [49m [43msafe_reraise [49m [43m( [49m [43m) [49m [43m: [49m
[32m 900 [39m [43m
[49m [43mpool [49m [43m. [49m [43mlogger [49m [43m. [49m [43mdebug [49m [43m( [49m [33;43m" [39;49m [33;43mError on
connect(): [39;49m [38;5;132;43;01m%s [39;49;00m [33;43m" [39;49m [43m, [49m [43m [49m [43me [49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/util/langhelpers.py:224 [39m, in [36msafe_reraise.__exit__ [39m [34m(self, type_, value,
traceback) [39m
[32m 223 [39m [38;5;28mself [39m._exc_info = [38;5;28;01mNone [39;00m [38;5;66;03m# remove potential
circular references [39;00m
[32m--> [39m [32m224 [39m [38;5;28;01mraise [39;00m exc_value.with_traceback(exc_tb)
[32m 225 [39m [38;5;28;01melse [39;00m:

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:895 [39m, in [36m_ConnectionRecord.__connect [39m [34m(self) [39m
[32m 894 [39m [38;5;28mself [39m.starttime = time.time()
[32m--> [39m [32m895 [39m [38;5;28mself [39m.dbapi_connection = connection =
[43mpool [49m [43m. [49m [43m_invoke_creator [49m [43m( [49m [38;5;28;43mself [39;49m [43m) [49m
[32m 896 [39m pool.logger.debug( [33m" [39m [33mCreated new connection [39m [38;5;132;01m%r [39;00m [33m" [39m,
connection)

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/create.py:661 [39m, in [36mcreate_engine.
<locals>.connect [39m [34m(connection_record) [39m
[32m 659 [39m [38;5;28;01mreturn [39;00m connection
[32m--> [39m [32m661 [39m [38;5;28;01mreturn [39;00m
[43mdialect [49m [43m. [49m [43mconnect [49m [43m( [49m [43m* [49m [43mcargs [49m [43m, [49m [43m
[49m [43m* [49m [43m* [49m [43mparams [49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/default.py:629 [39m, in [36mDefaultDialect.connect [39m [34m(self, *cargs,
**cparams) [39m

```

```
[32m      627 [39m [38;5;28;01mdef [39;00m [38;5;250m [39m [34mconnect [39m( [38;5;28mself [39m, *cargs: Any,
**cparams: Any) -> DBAPIDConnection:
[32m      628 [39m [38;5;66;03m# inherits the docstring from interfaces.Dialect.connect [39;00m
[32m--> [39m [32m629 [39m [38;5;28;01mreturn [39;00m
[38;5;28;43mself [39;49m [43m. [49m [43mloaded_dbapi [49m [43m. [49m [43mconnect [49m [43m( [49m [43m* [49m [43mcargs
[49m [43m, [49m [43m [49m [43m* [49m [43m* [49m [43mparams [49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/psycopg2/__init__.py:122 [39m, in [36mconnect [39m [34m(dsn, connection_factory, cursor_factory,
**kwargs) [39m
[32m      121 [39m dsn = _ext.make_dsn(dsn, **kwargs)
[32m--> [39m [32m122 [39m conn = [43mconnect [49m [43m( [49m [43mdsn [49m [43m, [49m [43m
[49m [43mconnection_factory [49m [43m= [49m [43mconnection_factory [49m [43m, [49m [43m
[49m [43m* [49m [43m* [49m [43mkwasync [49m [43m) [49m
[32m      123 [39m [38;5;28;01mif [39;00m cursor_factory [38;5;129;01mis [39;00m [38;5;129;01mnot [39;00m
[38;5;28;01mNone [39;00m:
```

```
[31mOperationalError [39m: connection to server at "localhost" (:::1), port 5432 failed: FATAL: role "username"
does not exist
```

The above exception was the direct cause of the following exception:

```
[31mOperationalError [39m                                Traceback (most recent call last)
[36mCell [39m [36m [39m [32mIn[35] [39m [32m, line 6 [39m
[32m      2 [39m [38;5;28;01mimport [39;00m [38;5;250m [39m [34;01mpandas [39;00m [38;5;250m
[39m [38;5;28;01mas [39;00m [38;5;250m [39m [34;01mpd [39;00m
[32m      4 [39m engine =
create_engine( [33m" [39m [33mpostgresql://username:password@localhost:5432/your_db [39m [33m" [39m)
[32m--> [39m [32m6 [39m df = [43mpd [49m [43m. [49m [43mread_sql [49m [43m( [49m [33;43m" [39;49m [33;43mSELECT *
FROM your_table; [39;49m [33;43m" [39;49m [43m, [49m [43m [49m [43mengine [49m [43m) [49m
[32m      7 [39m df.head()
```

```
[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-packages/pandas/io/sql.py:706 [39m, in
[36mread_sql [39m [34m(sql, con, index_col, coerce_float, params, parse_dates, columns, chunksize,
dtype_backend, dtype) [39m
[32m      703 [39m dtype_backend = [33m" [39m [33mnumpy [39m [33m" [39m [38;5;66;03m# type:
ignore[assignment] [39;00m
[32m      704 [39m [38;5;28;01massert [39;00m dtype_backend [38;5;129;01mis [39;00m [38;5;129;01mnot [39;00m
lib.no_default
[32m--> [39m [32m706 [39m [38;5;28;01mwith [39;00m [43mpandasSQL_builder [49m [43m( [49m [43mcon [49m [43m) [49m
[38;5;28;01mas [39;00m pandas_sql:
[32m      707 [39m [38;5;28;01mif [39;00m [38;5;28misinstance [39m(pandas_sql, SQLiteDatabase):
[32m      708 [39m [38;5;28;01mreturn [39;00m pandas_sql.read_query(
[32m      709 [39m sql,
[32m      710 [39m index_col=index_col,
[32m      (...) [39m [32m      716 [39m dtype=dtype,
[32m      717 [39m )
```

```
[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-packages/pandas/io/sql.py:908 [39m, in
[36mpandasSQL_builder [39m [34m(con, schema, need_transaction) [39m
[32m      905 [39m [38;5;28;01mraise [39;00m [38;5;167;01mImportError [39;00m( [33m" [39m [33mUsing URI string
without sqlalchemy installed. [39m [33m" [39m)
[32m      907 [39m [38;5;28;01mif [39;00m sqlalchemy [38;5;129;01mis [39;00m [38;5;129;01mnot [39;00m
[38;5;28;01mNone [39;00m [38;5;129;01mand [39;00m [38;5;28misinstance [39m(con, ( [38;5;28mstr [39m,
sqlalchemy.engine.Connectable)):
[32m--> [39m [32m908 [39m [38;5;28;01mreturn [39;00m
[43mSQLiteDatabase [49m [43m( [49m [43mcon [49m [43m, [49m [43m [49m [43mschema [49m [43m, [49m [43m
[49m [43mneed_transaction [49m [43m) [49m
[32m      910 [39m adbc = import_optional_dependency( [33m" [39m [33madbc_driver_manager.dbapi [39m [33m" [39m,
errors= [33m" [39m [33mignore [39m [33m" [39m)
[32m      911 [39m [38;5;28;01mif [39;00m adbc [38;5;129;01mand [39;00m [38;5;28misinstance [39m(con,
adbc.Connection):
```

```
[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-packages/pandas/io/sql.py:1648 [39m,
```

```

in [36mSQLDatabase.__init__ [39m [34m(self, con, schema, need_transaction) [39m
[32m 1646 [39m [38;5;28mself [39m.exit_stack.callback(con.dispose)
[32m 1647 [39m [38;5;28;01mif [39;00m [38;5;28misinstance [39m(con, Engine):
[32m-> [39m [32m1648 [39m con =
[38;5;28mself [39m.exit_stack.enter_context( [43mcon [49m [43m. [49m [43mconnect [49m [43m( [49m [43m) [49m)
[32m 1649 [39m [38;5;28;01mif [39;00m need_transaction [38;5;129;01mand [39;00m [38;5;129;01mnot [39;00m
con.in_transaction():
[32m 1650 [39m [38;5;28mself [39m.exit_stack.enter_context(con.begin())

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/base.py:3277 [39m, in [36mEngine.connect [39m [34m(self) [39m
[32m 3254 [39m [38;5;28;01mdef [39;00m [38;5;250m [39m [34mconnect [39m( [38;5;28mself [39m) -> Connection:
[32m 3255 [39m [38;5;250m [39m [33;03m""Return a new :class:`_engine.Connection` object. [39;00m
[32m 3256 [39m
[32m 3257 [39m [33;03m The :class:`_engine.Connection` acts as a Python context manager, so [39;00m
[32m (...) [39m [32m 3274 [39m
[32m 3275 [39m [33;03m "" [39;00m
[32m-> [39m [32m3277 [39m [38;5;28;01mreturn [39;00m
[38;5;28;43mself [39;49m [43m. [49m [43m_connection_cls [49m [43m( [49m [38;5;28;43mself [39;49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/base.py:145 [39m, in [36mConnection.__init__ [39m [34m(self, engine, connection,
_has_events, _allow_revalidate, _allow_autobegin) [39m
[32m 143 [39m [38;5;28mself [39m._dbapi_connection = engine.raw_connection()
[32m 144 [39m [38;5;28;01mexcept [39;00m dialect.loaded_dbapi.Error [38;5;28;01mas [39;00m err:
[32m--> [39m [32m145 [39m
[43mConnection [49m [43m. [49m [43m_handle_dbapi_exception_noconnection [49m [43m( [49m
[32m 146 [39m [43m [49m [43merr [49m [43m, [49m [43m [49m [43mdialect [49m [43m, [49m [43m
[49m [43mengine [49m
[32m 147 [39m [43m [49m [43m) [49m
[32m 148 [39m [38;5;28;01mraise [39;00m
[32m 149 [39m [38;5;28;01melse [39;00m:

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/base.py:2440 [39m, in
[36mConnection._handle_dbapi_exception_noconnection [39m [34m(cls, e, dialect, engine, is_disconnect,
invalidate_pool_on_disconnect, is_pre_ping) [39m
[32m 2438 [39m [38;5;28;01melif [39;00m should_wrap:
[32m 2439 [39m [38;5;28;01massert [39;00m sqlalchemy_exception [38;5;129;01mis [39;00m
[38;5;129;01mnot [39;00m [38;5;28;01mNone [39;00m
[32m-> [39m [32m2440 [39m [38;5;28;01mraise [39;00m
sqlalchemy_exception.with_traceback(exc_info[ [32m2 [39m]) [38;5;28;01mfrom [39;00m [38;5;250m
[39m [34;01me [39;00m
[32m 2441 [39m [38;5;28;01melse [39;00m:
[32m 2442 [39m [38;5;28;01massert [39;00m exc_info[ [32m1 [39m] [38;5;129;01mis [39;00m
[38;5;129;01mnot [39;00m [38;5;28;01mNone [39;00m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/base.py:143 [39m, in [36mConnection.__init__ [39m [34m(self, engine, connection,
_has_events, _allow_revalidate, _allow_autobegin) [39m
[32m 141 [39m [38;5;28;01mif [39;00m connection [38;5;129;01mis [39;00m [38;5;28;01mNone [39;00m:
[32m 142 [39m [38;5;28;01mtry [39;00m:
[32m--> [39m [32m143 [39m [38;5;28mself [39m._dbapi_connection =
[43mengine [49m [43m. [49m [43mraw_connection [49m [43m( [49m [43m) [49m
[32m 144 [39m [38;5;28;01mexcept [39;00m dialect.loaded_dbapi.Error [38;5;28;01mas [39;00m err:
[32m 145 [39m Connection._handle_dbapi_exception_noconnection(
[32m 146 [39m err, dialect, engine
[32m 147 [39m )

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/base.py:3301 [39m, in [36mEngine.raw_connection [39m [34m(self) [39m
[32m 3279 [39m [38;5;28;01mdef [39;00m [38;5;250m [39m [34mraw_connection [39m( [38;5;28mself [39m) ->
PoolProxiedConnection:
[32m 3280 [39m [38;5;250m [39m [33;03m""Return a "raw" DBAPI connection from the connection pool. [39;00m
[32m 3281 [39m

```

```

[32m 3282 [39m [33;03m The returned object is a proxied version of the DBAPI [39;00m
[32m (...) [39m [32m 3299 [39m
[32m 3300 [39m [33;03m "" [39;00m
[32m-> [39m [32m3301 [39m [38;5;28;01mreturn [39;00m
[38;5;28;43mself [39;49m [43m. [49m [43mpool [49m [43m. [49m [43mconnect [49m [43m( [49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:447 [39m, in [36mPool.connect [39m [34m(self) [39m
[32m 439 [39m [38;5;28;01mdef [39;00m [38;5;250m [39m [34mconnect [39m( [38;5;28mself [39m) ->
PoolProxiedConnection:
[32m 440 [39m [38;5;250m [39m [33;03m""Return a DBAPI connection from the pool. [39;00m
[32m 441 [39m
[32m 442 [39m [33;03m The connection is instrumented such that when its [39;00m
[32m (...) [39m [32m 445 [39m
[32m 446 [39m [33;03m "" [39;00m
[32m--> [39m [32m447 [39m [38;5;28;01mreturn [39;00m
[43m_ConnectionFairy [49m [43m. [49m [43m_checkout [49m [43m( [49m [38;5;28;43mself [39;49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:1264 [39m, in [36m_ConnectionFairy._checkout [39m [34m(cls, pool, threadconns,
fairy) [39m
[32m 1256 [39m [38;5;129m@classmethod [39m
[32m 1257 [39m [38;5;28;01mdef [39;00m [38;5;250m [39m [34m_checkout [39m(
[32m 1258 [39m [38;5;28mcls [39m,
[32m (...) [39m [32m 1261 [39m fairy: Optional[_ConnectionFairy] = [38;5;28;01mNone [39;00m,
[32m 1262 [39m ) -> _ConnectionFairy:
[32m 1263 [39m [38;5;28;01mif [39;00m [38;5;129;01mnot [39;00m fairy:
[32m-> [39m [32m1264 [39m fairy =
[43m_ConnectionRecord [49m [43m. [49m [43m_checkout [49m [43m( [49m [43mpool [49m [43m) [49m
[32m 1266 [39m [38;5;28;01mif [39;00m threadconns [38;5;129;01mis [39;00m [38;5;129;01mnot [39;00m
[38;5;28;01mNone [39;00m:
[32m 1267 [39m threadconns.current = weakref.ref(fairy)

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:711 [39m, in [36m_ConnectionRecord.checkout [39m [34m(cls, pool) [39m
[32m 709 [39m rec = cast(_ConnectionRecord, pool._do_get())
[32m 710 [39m [38;5;28;01melse [39;00m:
[32m--> [39m [32m711 [39m rec = [43mpool [49m [43m. [49m [43m_do_get [49m [43m( [49m [43m) [49m
[32m 713 [39m [38;5;28;01mtry [39;00m:
[32m 714 [39m dbapi_connection = rec.get_connection()

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/impl.py:177 [39m, in [36mQueuePool._do_get [39m [34m(self) [39m
[32m 175 [39m [38;5;28;01mreturn [39;00m [38;5;28mself [39m._create_connection()
[32m 176 [39m [38;5;28;01mexcept [39;00m:
[32m--> [39m [32m177 [39m [43m [49m [38;5;28;43;01mwith [39;49;00m [43m
[49m [43mutil [49m [43m. [49m [43msafe_reraise [49m [43m( [49m [43m) [49m [43m: [49m
[32m 178 [39m [43m [49m [38;5;28;43mself [39;49m [43m. [49m [43m_dec_overflow [49m [43m( [49m [43m) [49m
[32m 179 [39m [38;5;28;01mraise [39;00m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/util/langhelpers.py:224 [39m, in [36msafe_reraise.__exit__ [39m [34m(self, type_, value,
traceback) [39m
[32m 222 [39m [38;5;28;01massert [39;00m exc_value [38;5;129;01mis [39;00m [38;5;129;01mnot [39;00m
[38;5;28;01mNone [39;00m
[32m 223 [39m [38;5;28mself [39m._exc_info = [38;5;28;01mNone [39;00m [38;5;66;03m# remove potential
circular references [39;00m
[32m--> [39m [32m224 [39m [38;5;28;01mraise [39;00m exc_value.with_traceback(exc_tb)
[32m 225 [39m [38;5;28;01melse [39;00m:
[32m 226 [39m [38;5;28mself [39m._exc_info = [38;5;28;01mNone [39;00m [38;5;66;03m# remove potential
circular references [39;00m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/impl.py:175 [39m, in [36mQueuePool._do_get [39m [34m(self) [39m
[32m 173 [39m [38;5;28;01mif [39;00m [38;5;28mself [39m._inc_overflow():

```

```

[32m 174 [39m [38;5;28;01mtry [39;00m:
[32m--> [39m [32m175 [39m [38;5;28;01mreturn [39;00m
[38;5;28;43mself [39;49m [43m. [49m [43m_create_connection [49m [43m( [49m [43m) [49m
[32m 176 [39m [38;5;28;01mexcept [39;00m:
[32m 177 [39m [38;5;28;01mwith [39;00m util.safe_reraise():

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:388 [39m, in [36mPool._create_connection [39m [34m(self) [39m
[32m 385 [39m [38;5;28;01mdef [39;00m [38;5;250m [39m [34m_create_connection [39m( [38;5;28mself [39m) ->
ConnectionPoolEntry:
[32m 386 [39m [38;5;250m [39m [33;03m"""Called by subclasses to create a new ConnectionRecord.""" [39;00m
[32m--> [39m [32m388 [39m [38;5;28;01mreturn [39;00m
[43m_ConnectionRecord [49m [43m( [49m [38;5;28;43mself [39;49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:673 [39m, in [36m_ConnectionRecord.__init__ [39m [34m(self, pool, connect) [39m
[32m 671 [39m [38;5;28mself [39m.__pool = pool
[32m 672 [39m [38;5;28;01mif [39;00m connect:
[32m--> [39m [32m673 [39m [38;5;28;43mself [39;49m [43m. [49m [43m__connect [49m [43m( [49m [43m) [49m
[32m 674 [39m [38;5;28mself [39m.finalize_callback = deque()

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:899 [39m, in [36m_ConnectionRecord.__connect [39m [34m(self) [39m
[32m 897 [39m [38;5;28mself [39m.fresh = [38;5;28;01mTrue [39;00m
[32m 898 [39m [38;5;28;01mexcept [39;00m [38;5;167;01mBaseException [39;00m [38;5;28;01mas [39;00m e:
[32m--> [39m [32m899 [39m [43m [49m [38;5;28;43;01mwith [39;49;00m [43m
[49m [43mutil [49m [43m. [49m [43msafe_reraise [49m [43m( [49m [43m) [49m [43m: [49m
[32m 900 [39m [43m
[49m [43mpool [49m [43m. [49m [43mlogger [49m [43m. [49m [43mdebug [49m [43m( [49m [33;43m" [39;49m [33;43mError on
connect(): [39;49m [38;5;132;43;01m%s [39;49;00m [33;43m" [39;49m [43m, [49m [43m [49m [43me [49m [43m) [49m
[32m 901 [39m [38;5;28;01melse [39;00m:
[32m 902 [39m [38;5;66;03m# in SQLAlchemy 1.4 the first_connect event is not used by [39;00m
[32m 903 [39m [38;5;66;03m# the engine, so this will usually not be set [39;00m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/util/langhelpers.py:224 [39m, in [36msafe_reraise.__exit__ [39m [34m(self, type_, value,
traceback) [39m
[32m 222 [39m [38;5;28;01massert [39;00m exc_value [38;5;129;01mis [39;00m [38;5;129;01mnot [39;00m
[38;5;28;01mNone [39;00m
[32m 223 [39m [38;5;28mself [39m._exc_info = [38;5;28;01mNone [39;00m [38;5;66;03m# remove potential
circular references [39;00m
[32m--> [39m [32m224 [39m [38;5;28;01mraise [39;00m exc_value.with_traceback(exc_tb)
[32m 225 [39m [38;5;28;01melse [39;00m:
[32m 226 [39m [38;5;28mself [39m._exc_info = [38;5;28;01mNone [39;00m [38;5;66;03m# remove potential
circular references [39;00m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/pool/base.py:895 [39m, in [36m_ConnectionRecord.__connect [39m [34m(self) [39m
[32m 893 [39m [38;5;28;01mtry [39;00m:
[32m 894 [39m [38;5;28mself [39m.starttime = time.time()
[32m--> [39m [32m895 [39m [38;5;28mself [39m.dbapi_connection = connection =
[43mpool [49m [43m. [49m [43m_invoke_creator [49m [43m( [49m [38;5;28;43mself [39;49m [43m) [49m
[32m 896 [39m pool.logger.debug( [33m" [39m [33mCreated new connection
[39m [38;5;132;01m%r [39;00m [33m" [39m, connection)
[32m 897 [39m [38;5;28mself [39m.fresh = [38;5;28;01mTrue [39;00m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/create.py:661 [39m, in [36mcreate_engine.
<locals>.connect [39m [34m(connection_record) [39m
[32m 658 [39m [38;5;28;01mif [39;00m connection [38;5;129;01mis [39;00m [38;5;129;01mnot [39;00m
[38;5;28;01mNone [39;00m:
[32m 659 [39m [38;5;28;01mreturn [39;00m connection
[32m--> [39m [32m661 [39m [38;5;28;01mreturn [39;00m
[43mdialect [49m [43m. [49m [43mconnect [49m [43m( [49m [43m* [49m [43mcargs [49m [43m, [49m [43m
[49m [43m* [49m [43m* [49m [43mparams [49m [43m) [49m

```

```
[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/sqlalchemy/engine/default.py:629 [39m, in [36mDefaultDialect.connect [39m [34m(self, *cargs,
**cparams) [39m
[32m     627 [39m     [38;5;28;01mdef [39;00m [38;5;250m [39m [34mconnect [39m [34m(self, *cargs: Any,
**cparams: Any) -> DBAPIDialectConnection:
[32m     628 [39m         [38;5;66;03m# inherits the docstring from interfaces.Dialect.connect [39;00m
[32m--> [39m [32m629 [39m         [38;5;28;01mreturn [39;00m
[38;5;28;43mself [39;49m [43m. [49m [43mloaded_dbapi [49m [43m. [49m [43mconnect [49m [43m [49m [43m* [49m [43mcargs
[49m [43m, [49m [43m [49m [43m* [49m [43m* [49m [43mcparams [49m [43m) [49m

[36mFile [39m [32m~/Desktop/MSBA Q1/Regression/.venv/lib/python3.11/site-
packages/psycopg2/__init__.py:122 [39m, in [36mconnect [39m [34m(dsn, connection_factory, cursor_factory,
**kwargs) [39m
[32m     119 [39m         kwasync[ [33m' [39m [33masync_ [39m [33m' [39m] =
kwargs.pop( [33m' [39m [33masync_ [39m [33m' [39m)
[32m     121 [39m     dsn = _ext.make_dsn(dsn, **kwargs)
[32m--> [39m [32m122 [39m     conn = [43m_connect [49m [43m [49m [43mdsn [49m [43m, [49m [43m
[49m [43mconnection_factory [49m [43m= [49m [43mconnection_factory [49m [43m, [49m [43m
[49m [43m* [49m [43m* [49m [43mkwasync [49m [43m) [49m
[32m     123 [39m     [38;5;28;01mif [39;00m cursor_factory [38;5;129;01mis [39;00m [38;5;129;01mnot [39;00m
[38;5;28;01mNone [39;00m:
[32m     124 [39m         conn.cursor_factory = cursor_factory

[31mOperationalError [39m: (psycopg2.OperationalError) connection to server at "localhost" (:::1), port 5432
failed: FATAL:  role "username" does not exist

(Background on this error at: https://sqlalche.me/e/20/e3q8)
```

```
engine = create_engine("postgres://postgres:your_password@localhost:5432/Entertainment")
```

```
from sqlalchemy import inspect
import pandas as pd

insp = inspect(engine)
tables = insp.get_table_names()
print(tables)
```

```
['agents', 'customers', 'engagements', 'entertainer_members', 'entertainer_styles', 'entertainers', 'members',
'musical_preferences', 'musical_styles', 'ztbldays', 'ztblmonths', 'ztblskiplabels', 'ztblweeks']
```

```
df_movies = pd.read_sql("SELECT * FROM agents;", engine) # replace with your table name
df_movies.head() # see the first few rows
```

	agent id	agtfirsrn ame	agtlastna me	agtstreetaddres s	agtcit y	agtsta te	agtzipc ode	agtphonenu mber	datehire d	salar y	commission rate
0	1	William	Thompso n	122 Spring River Drive	Redmo nd	WA	98053	555-2681	1997- 05-15	3500 0.0	0.040
1	2	Scott	Bishop	66 Spring Valley Drive	Seattle	WA	98125	555-2666	1998- 02-05	27000 .0	0.040
2	3	Carol	Viescas	667 Red River Road	Bellevu e	WA	98006	555-2571	1997-11- 19	3000 0.0	0.050
3	4	Karen	Smith	30301 - 166th Ave. N.E.	Seattle	WA	98125	555-2551	1998- 03-05	2200 0.0	0.055
4	5	Marianne	Wier	908 W. Capital Way	Tacom a	WA	98413	555-2606	1998- 02-02	2450 0.0	0.045


```
dfs = {t: pd.read_sql(f"SELECT * FROM {t};", engine) for t in tables}
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Merge entertainers with agents
df_eng = dfs["engagements"].merge(dfs["agents"], left_on="agentid", right_on="agentid", how="left")
df_eng = df_eng.merge(dfs["entertainers"], left_on="entertainerid", right_on="entertainerid", how="left")
df_eng["agent_full_name"] = df_eng["agtfirstname"] + " " + df_eng["agtlastname"]

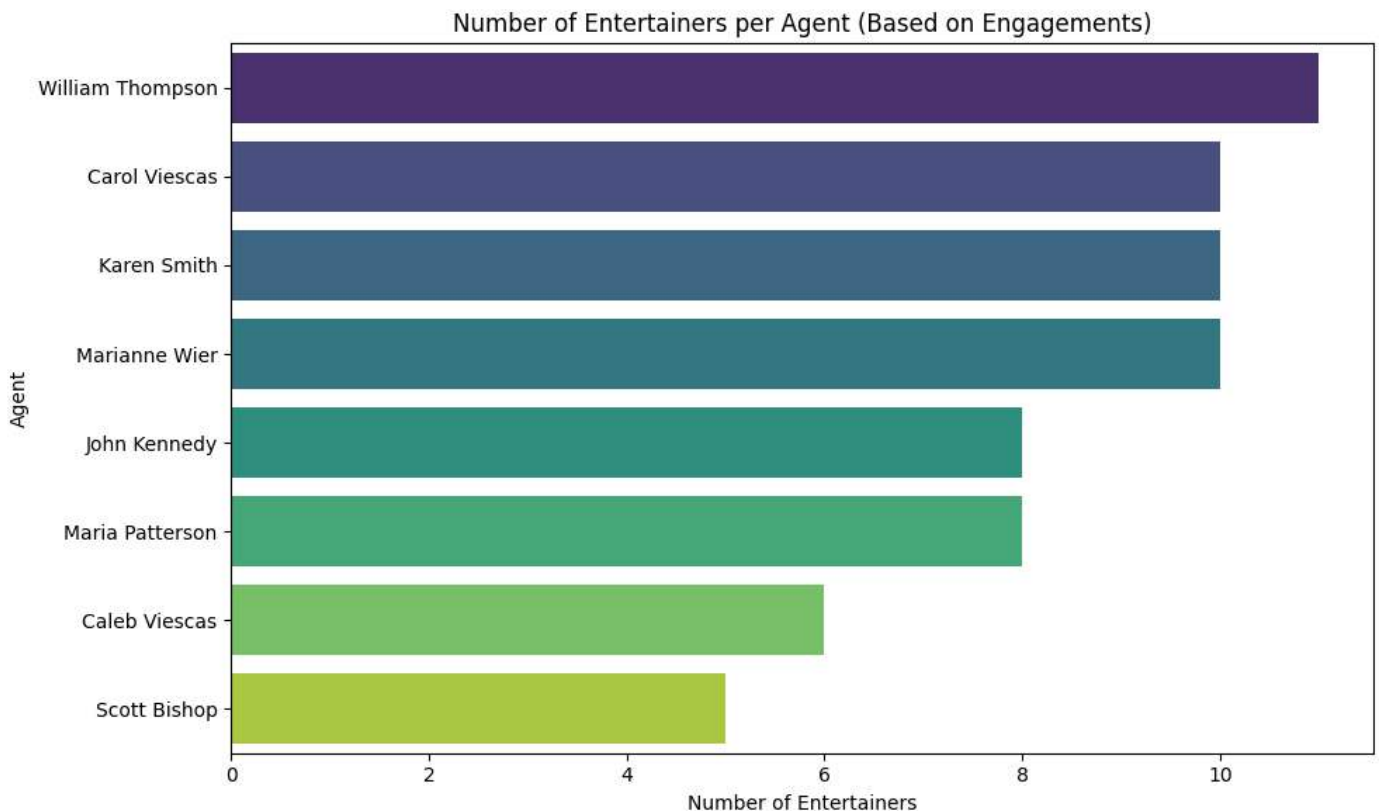
# Count unique entertainers per agent based on engagements
ent_counts = df_eng.groupby("agent_full_name")["entertainerid"].nunique().sort_values(ascending=False)

# Plot
plt.figure(figsize=(10,6))
sns.barplot(x=ent_counts.values, y=ent_counts.index, palette="viridis")
plt.title("Number of Entertainers per Agent (Based on Engagements)")
plt.xlabel("Number of Entertainers")
plt.ylabel("Agent")
plt.tight_layout()
plt.show()
```

```
/var/folders/k1/g0334f897ldbhywx15n88j900000gn/T/ipykernel_28746/162434391.py:16: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=ent_counts.values, y=ent_counts.index, palette="viridis")
```



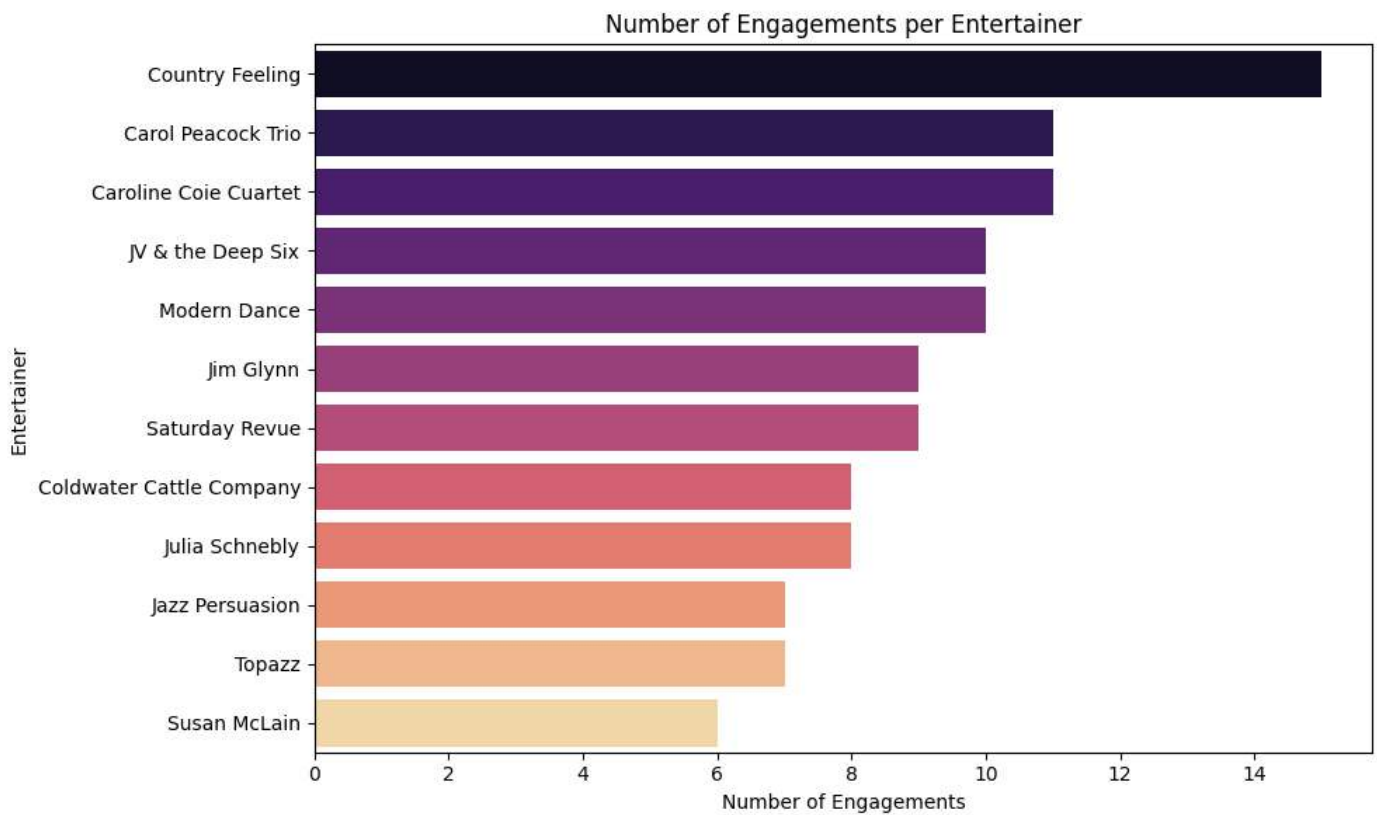
```
eng_counts = df_eng.groupby("entstagename")["engagementnumber"].count().sort_values(ascending=False)

# Plot
plt.figure(figsize=(10,6))
sns.barplot(x=eng_counts.values, y=eng_counts.index, palette="magma")
plt.title("Number of Engagements per Entertainer")
plt.xlabel("Number of Engagements")
plt.ylabel("Entertainer")
plt.tight_layout()
plt.show()
```

/var/folders/k1/g0334f897ldbhywx15n88j900000gn/T/ipykernel_28746/18646894.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=eng_counts.values, y=eng_counts.index, palette="magma")
```



```
dfs['entertainers'].head()
dfs['engagements'].head()
```

	engagementnumber	startdate	enddate	starttime	stoptime	contractprice	customerid	agentid	entertainerid
0	2	2017-09-02	2017-09-06	13:00:00	15:00:00	200.0	10006	4	1004
1	3	2017-09-11	2017-09-16	13:00:00	15:00:00	590.0	10001	3	1005
2	4	2017-09-12	2017-09-18	20:00:00	00:00:00	470.0	10007	3	1004
3	5	2017-09-12	2017-09-15	16:00:00	19:00:00	1130.0	10006	5	1003
4	6	2017-09-11	2017-09-15	15:00:00	21:00:00	2300.0	10014	7	1008

```

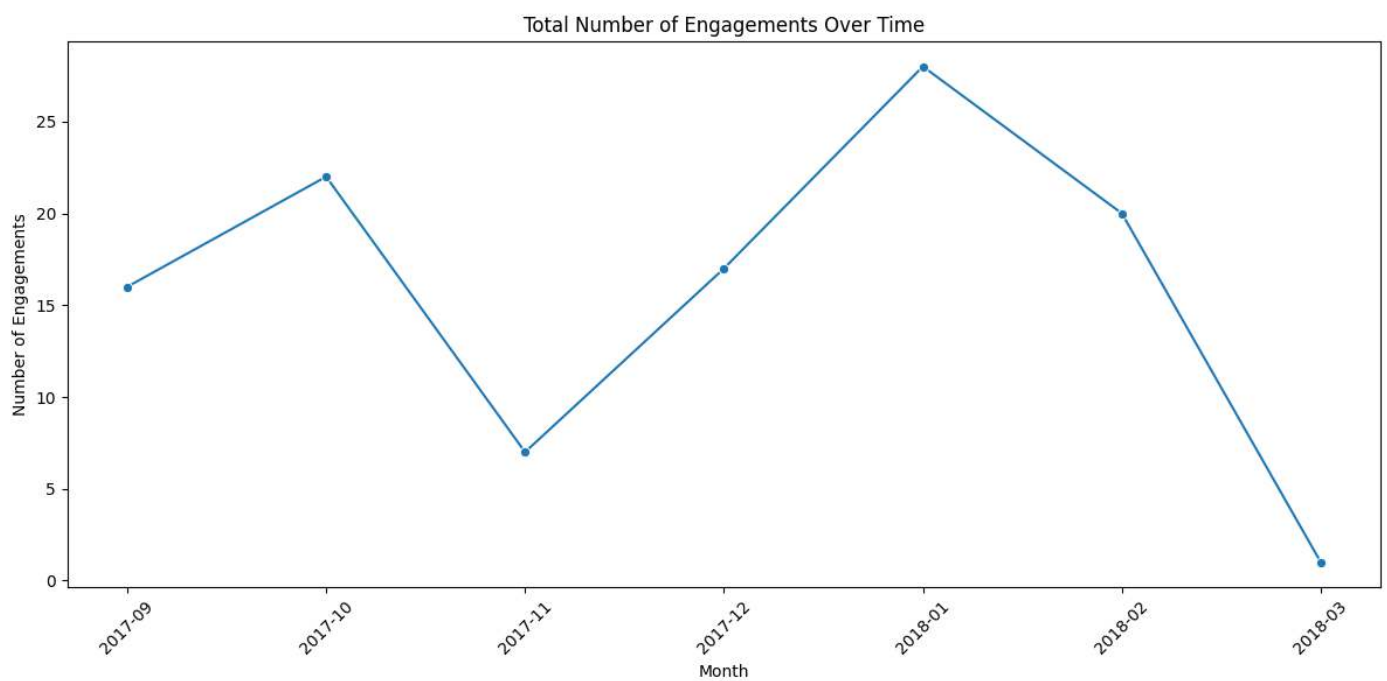
df_eng['startdate'] = pd.to_datetime(df_eng['startdate'])

# Extract month and year
df_eng['year_month'] = df_eng['startdate'].dt.to_period('M')

# Count engagements per month
monthly_counts = df_eng.groupby('year_month').size()

# Plot
plt.figure(figsize=(12,6))
sns.lineplot(x=monthly_counts.index.astype(str), y=monthly_counts.values, marker='o')
plt.title("Total Number of Engagements Over Time")
plt.xlabel("Month")
plt.ylabel("Number of Engagements")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



```
df_eng.head()
```

	engagement number	startdate	enddate	starttime	stop time	contract price	customer id	agent id	enter tainer id	agent first name	. . .	ent ssn	entst reeta ddress	en tci ty	en tst ate	en tzi pc ode	entp hon enu mber	ent web page	ente mail address	dat ee nte red	age nt_ full_ name
0	2	2017-09-02	2017-09-06	13:00:00	15:00:00	200.0	10006	4	1004	Karen	. . .	888-26-1025	13920 S.E. 40th Street	Bellevue	WA	98009	555-2531	None	None	1996-04-01	Karen Smith
1	3	2017-09-11	2017-09-16	13:00:00	15:00:00	590.0	10001	3	1005	Carol	. . .	888-30-1031	233 West Valley Hwy	Bellevue	WA	98005	555-2541	www.jazzperr.com	None	1997-05-12	Carol Viescas
2	4	2017-09-12	2017-09-18	20:00:00	00:00:00	470.0	10007	3	1004	Carol	. . .	888-26-1025	13920 S.E. 40th Street	Bellevue	WA	98009	555-2531	None	None	1996-04-01	Carol Viescas
3	5	2017-09-12	2017-09-15	16:00:00	19:00:00	1130.0	10006	5	1003	Mariann	. . .	888-18-1013	15127 NE 24th, #383	Redmond	WA	98052	555-2511	www.jvd6.com	jv@myspring.com	1998-03-18	Mariann Wier
4	6	2017-09-11	2017-09-15	15:00:00	21:00:00	2300.0	10014	7	1008	Caleb	. . .	888-98-1133	PO Box 223311	Seattle	WA	98125	555-2711	None	None	1996-02-28	Caleb Viescas

5 rows x 30 columns

```

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Ensure startdate is datetime
df_eng['startdate'] = pd.to_datetime(df_eng['startdate'])

# Extract month and day of week
df_eng['month'] = df_eng['startdate'].dt.month_name()
df_eng['day_of_week'] = df_eng['startdate'].dt.day_name()

# Pivot table: rows = day_of_week, columns = month, values = engagement counts
heat_data = df_eng.pivot_table(
    index='day_of_week',
    columns='month',
    values='engagementnumber', # any column; we count occurrences
    aggfunc='count'
).fillna(0) # fill NaN with 0

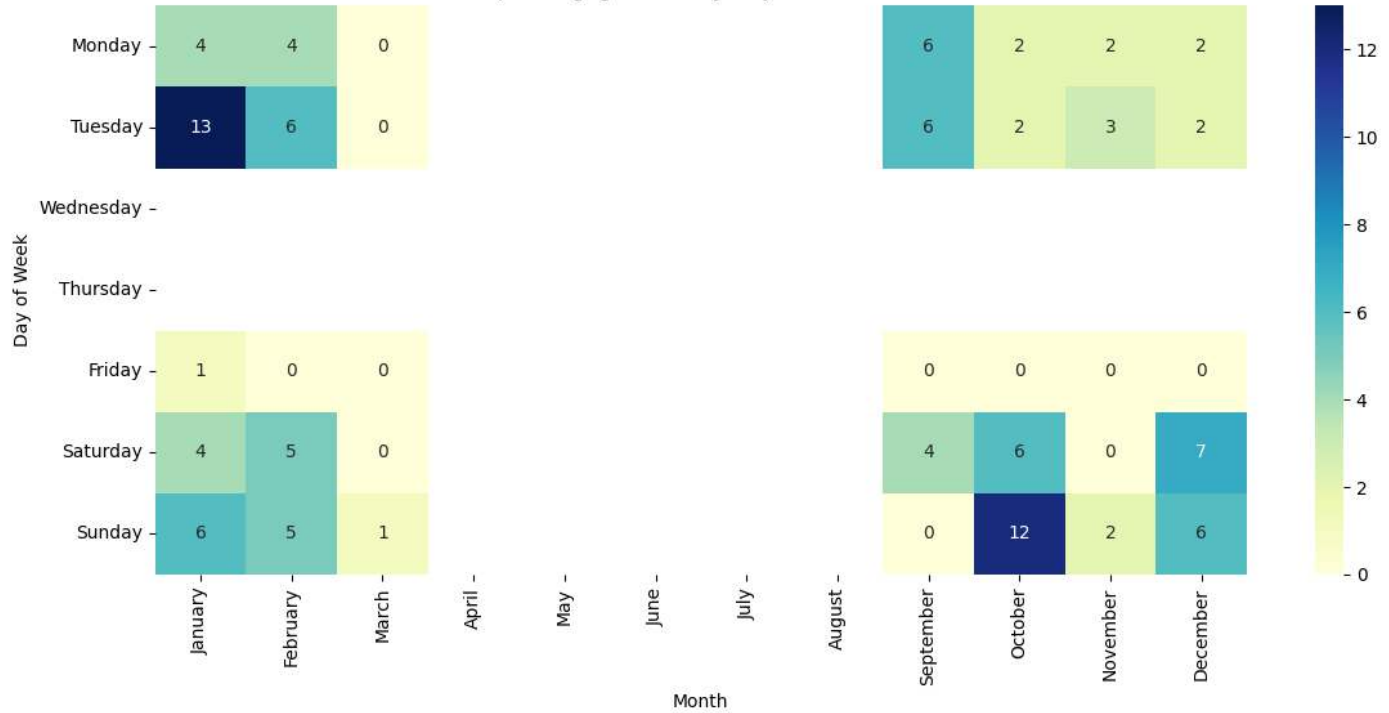
# Reorder rows and columns
month_order = ["January", "February", "March", "April", "May", "June",
               "July", "August", "September", "October", "November", "December"]
day_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]

heat_data = heat_data.reindex(index=day_order, columns=month_order)

# Plot heatmap
plt.figure(figsize=(12,6))
sns.heatmap(
    heat_data.values, # pass plain NumPy array
    annot=heat_data.values, # annotate with same values
    fmt=".0f", # round floats to integer
    cmap="YlGnBu",
    xticklabels=month_order,
    yticklabels=day_order
)
plt.title("Heatmap of Engagements by Day of Week and Month")
plt.xlabel("Month")
plt.ylabel("Day of Week")
plt.tight_layout()
plt.show()

```

Heatmap of Engagements by Day of Week and Month

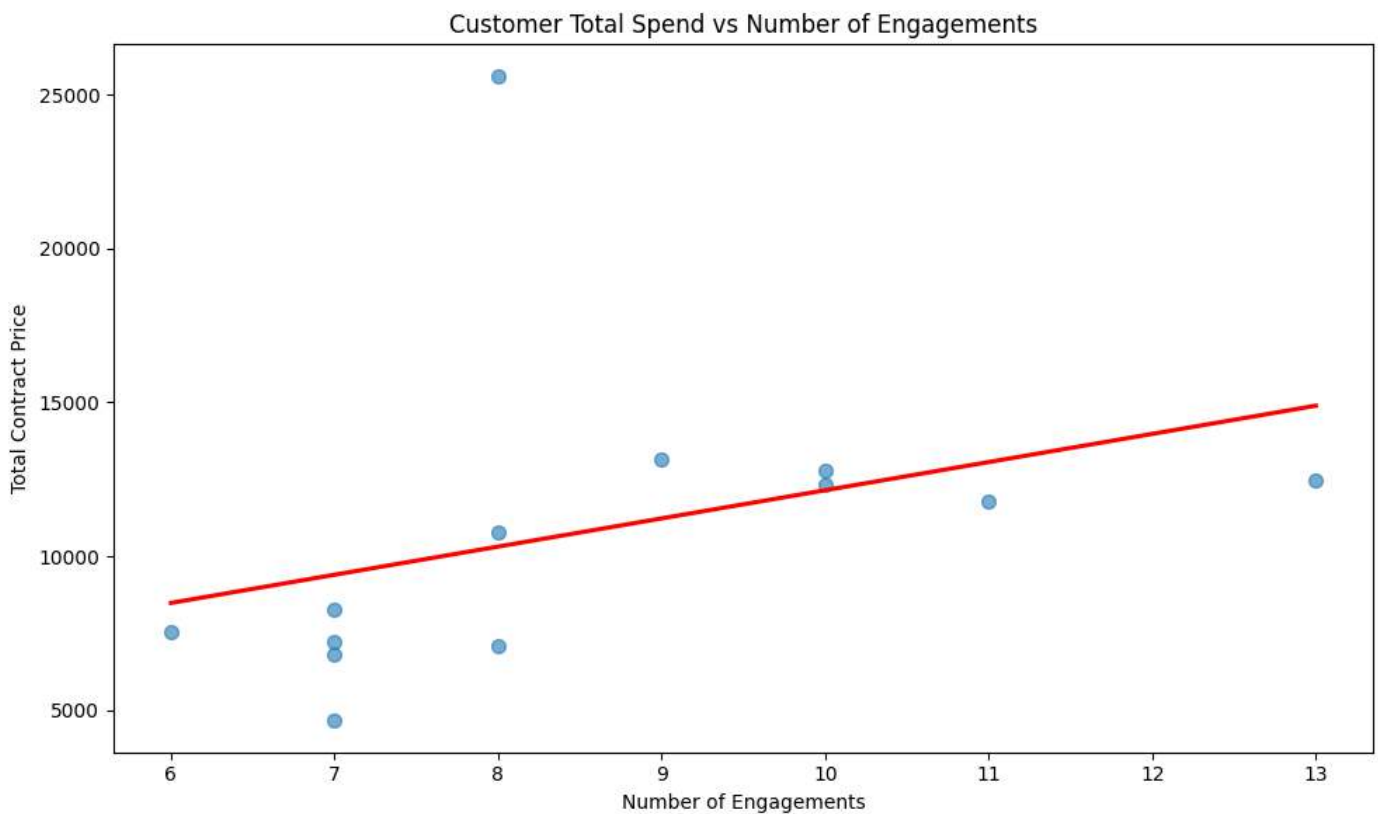


```

customer_stats = df_eng.groupby('customerid').agg({
    'engagementnumber': 'count',    # number of engagements
    'contractprice': 'sum'          # total contract price
}).reset_index()

# Scatter plot with fitted regression line
plt.figure(figsize=(10,6))
sns.regplot(
    x='engagementnumber',
    y='contractprice',
    data=customer_stats,
    scatter_kws={'s':50, 'alpha':0.6},
    line_kws={'color':'red'},
    ci=None
)
plt.xlabel('Number of Engagements')
plt.ylabel('Total Contract Price')
plt.title('Customer Total Spend vs Number of Engagements')
plt.tight_layout()
plt.show()

```



```

df_entertainer_styles = dfs['entertainer_styles'].merge(
    dfs['musical_styles'],
    on='styleid',          # replace with the actual column linking styles
    how='left'
)

# Optional: merge with entertainers to get entertainer names
df_entertainer_styles = df_entertainer_styles.merge(
    dfs['entertainers'],
    on='entertainerid',    # replace with your actual key
    how='left'
)

# Inspect
df_entertainer_styles.head()

```

	entertainerid	styleid	styles trength	stylename	entstage name	entssn	entstreet address	entcity	entstate	entzipcode	entphonenumber	entweb page	entemail address	dateentered
0	1001	10	2	Contemporary	Carol Peacock Trio	888-90-1121	4110 Old Redmond Rd.	Redmond	WA	98052	555-2691	www.cptrio.com	carolp@cptrio.com	1997-05-24
1	1001	20	1	Show Tunes	Carol Peacock Trio	888-90-1121	4110 Old Redmond Rd.	Redmond	WA	98052	555-2691	www.cptrio.com	carolp@cptrio.com	1997-05-24
2	1001	21	3	Standards	Carol Peacock Trio	888-90-1121	4110 Old Redmond Rd.	Redmond	WA	98052	555-2691	www.cptrio.com	carolp@cptrio.com	1997-05-24
3	1002	17	2	Motown	Topazz	888-50-1061	16 Maple Lane	Auburn	WA	98002	555-2591	www.topazz.com	None	1996-02-14
4	1002	19	1	Rhythm and Blues	Topazz	888-50-1061	16 Maple Lane	Auburn	WA	98002	555-2591	www.topazz.com	None	1996-02-14


```

df_style = df_eng.merge(df_entertainer_styles, on='entertainerid', how='left')

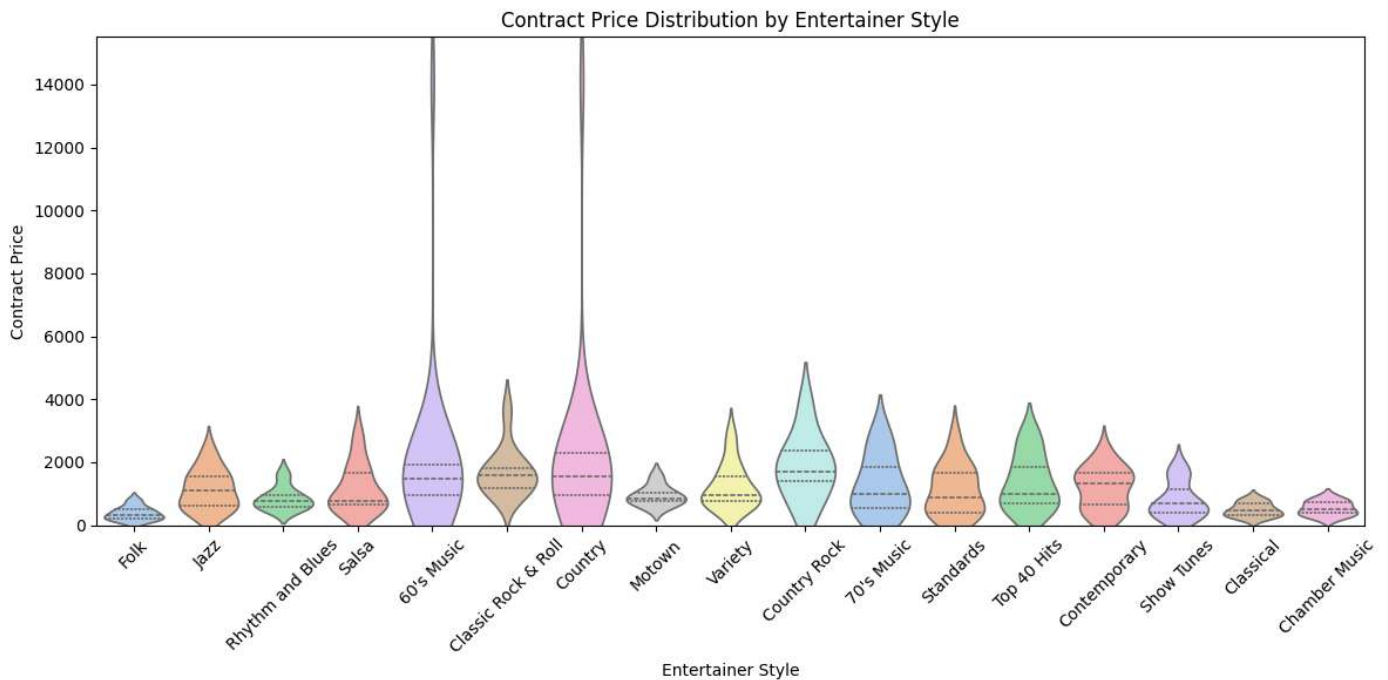
plt.figure(figsize=(12,6))
sns.violinplot(
    x='stylename', # your style column
    y='contractprice',
    data=df_style,
    inner='quartile',
    palette='pastel'
)
plt.ylim(0, df_style['contractprice'].max() * 1.1) # start Y-axis at 0
plt.xticks(rotation=45)
plt.xlabel('Entertainer Style')
plt.ylabel('Contract Price')
plt.title('Contract Price Distribution by Entertainer Style')
plt.tight_layout()
plt.show()

```

/var/folders/kl/g0334f897ldbhywx15n88j900000gn/T/ipykernel_28746/829610002.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.violinplot(
```



```

df_customer_styles = dfs['musical_preferences'].merge(
    dfs['musical_styles'],
    left_on='styleid', # adjust if your column is named differently
    right_on='styleid',
    how='left'
)

# Merge with customers to get customer names
df_customer_styles = df_customer_styles.merge(
    dfs['customers'],
    on='customerid', # adjust if your column is named differently
    how='left'
)

# Optional: create a full name column for convenience
df_customer_styles['customer_name'] = df_customer_styles['custfirstname'] + ' ' +
df_customer_styles['custlastname']

# Inspect
df_customer_styles.head()

```

	custo merid	styl eid	preferen ceseq	stylename	custfirst name	custlast name	custstreeta ddress	cust city	custs tate	custzip code	custphone number	customer _name
0	10001	10	2	Contempor ary	Doris	Hartwig	4726 - 11th Ave. N.E.	Seatt le	WA	98105	555-2671	Doris Hartwig
1	10001	22	1	Top 40 Hits	Doris	Hartwig	4726 - 11th Ave. N.E.	Seatt le	WA	98105	555-2671	Doris Hartwig
2	10002	3	1	60's Music	Deb	Waldal	908 W. Capital Way	Taco ma	WA	98413	555-2496	Deb Waldal
3	10002	8	2	Classic Rock & Roll	Deb	Waldal	908 W. Capital Way	Taco ma	WA	98413	555-2496	Deb Waldal
4	10003	17	2	Motown	Peter	Brehm	722 Moss Bay Blvd.	Kirkla nd	WA	98033	555-2501	Peter Brehm

```

style_counts = df_customer_styles['stylename'].value_counts()

plt.figure(figsize=(12,6))
sns.barplot(x=style_counts.index, y=style_counts.values, palette='pastel')
plt.xticks(rotation=45)
plt.xlabel('Musical Style')
plt.ylabel('Number of Customers')
plt.title('Number of Customers per Musical Style')
plt.tight_layout()
plt.show()

```

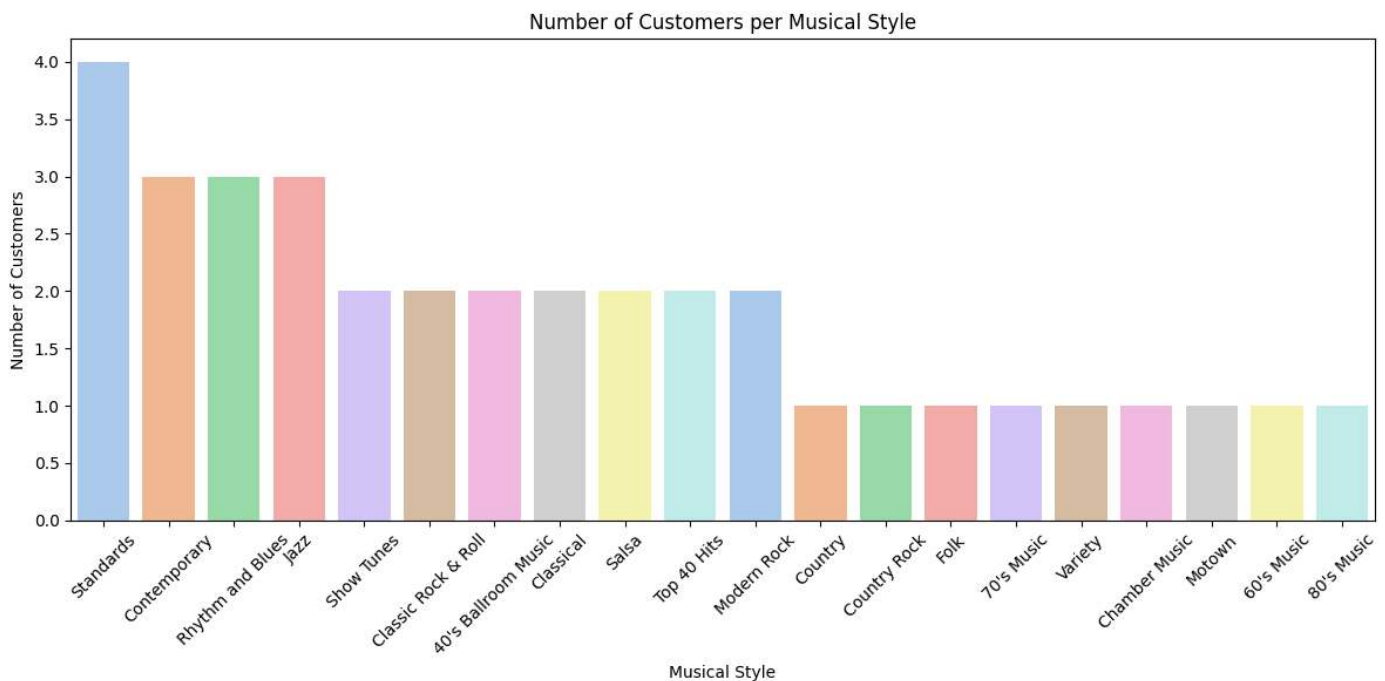
/var/folders/k1/g0334f8971dbhywx15n88j900000gn/T/ipykernel_28746/1611800508.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.barplot(x=style_counts.index, y=style_counts.values, palette='pastel')

```



```

## Agents by contract size
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams.update({
    'font.size': 16,
    'axes.titlesize': 20,
    'axes.labelsize': 18,
    'xtick.labelsize': 14,
    'ytick.labelsize': 14,
})

# --- Merge the data ---
df_eng = dfs["engagements"].merge(dfs["agents"], on="agentid", how="left")
df_eng = df_eng.merge(dfs["entertainers"], on="entertainerid", how="left")
df_eng["agent_full_name"] = df_eng["agtfirstname"] + " " + df_eng["agtlastname"]

# --- Compute contract totals per agent using contractprice ---
contract_totals = df_eng.groupby("agent_full_name")["contractprice"].sum()

# --- Pull agent salaries ---
agent_salaries = df_eng.groupby("agent_full_name")["salary"].first()

# --- Combine into a dataframe ---
df_plot = pd.DataFrame({
    "contract_total": contract_totals,
    "salary": agent_salaries
})

# Sort by contract totals (optional)
df_plot = df_plot.sort_values("contract_total", ascending=False)

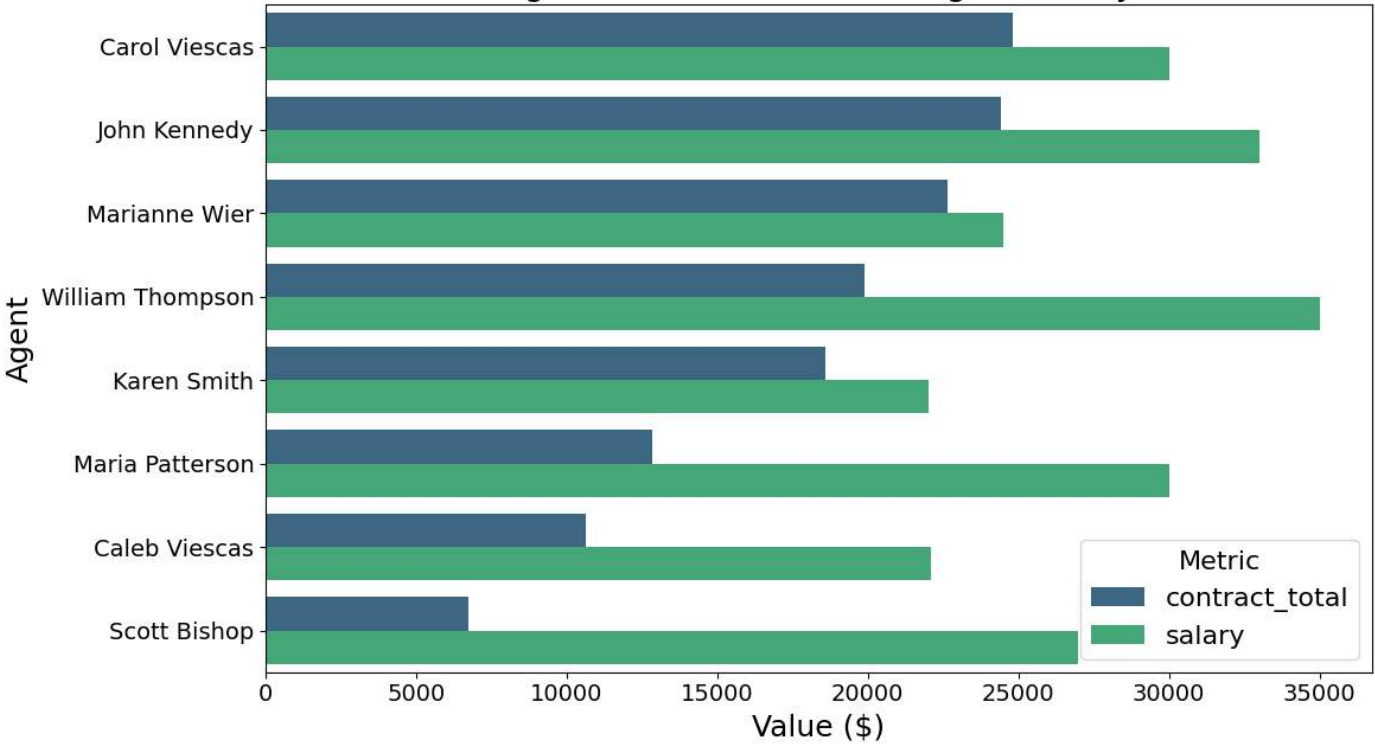
# --- Melt for side-by-side barplot ---
df_melt = df_plot.reset_index().melt(
    id_vars="agent_full_name",
    value_vars=["contract_total", "salary"],
    var_name="metric",
    value_name="value"
)

# --- Plot ---
plt.figure(figsize=(12,7))
sns.barplot(
    data=df_melt,
    x="value",
    y="agent_full_name",
    hue="metric",
    palette="viridis"
)

plt.title("Agent Contract Totals vs. Agent Salary")
plt.xlabel("Value ($)")
plt.ylabel("Agent")
plt.legend(title="Metric")
plt.tight_layout()
plt.show()

```

Agent Contract Totals vs. Agent Salary



```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# --- Aggregate per entertainer ---
perf_stats = (
    df_eng.groupby("entstagename")
        .agg(
            engagements=("engagementnumber", "count"),
            avg_revenue=("contractprice", "mean")
        )
        .reset_index()
)

# --- Larger text settings ---
plt.rcParams.update({
    'font.size': 16,
    'axes.titlesize': 20,
    'axes.labelsize': 18,
    'xtick.labelsize': 16,
    'ytick.labelsize': 16,
})

# --- Scatterplot ---
plt.figure(figsize=(12,8))
sns.scatterplot(
    x=perf_stats["engagements"],
    y=perf_stats["avg_revenue"],
    s=120,
    color='steelblue', # all points blue
    alpha=0.8
)

# Identify top performer
top_performer = perf_stats.sort_values("engagements", ascending=False).head(1)
other_top = perf_stats.sort_values("engagements", ascending=False).iloc[1:5]

# Label top performer in green
for _, row in top_performer.iterrows():
    plt.text(
        row["engagements"] + 0.1,
        row["avg_revenue"] + 5,
        row["entstagename"],
        fontsize=14,
        color='green'
    )

# Label next top performers in black
for _, row in other_top.iterrows():
    plt.text(
        row["engagements"] + 0.1,
        row["avg_revenue"] + 5,
        row["entstagename"],
        fontsize=14,
        color='black'
    )

# Label lowest performer
lowest = perf_stats.sort_values("engagements", ascending=True).head(1)
for _, row in lowest.iterrows():
    plt.text(
        row["engagements"] + 0.1,
        row["avg_revenue"] - 5,
        row["entstagename"],

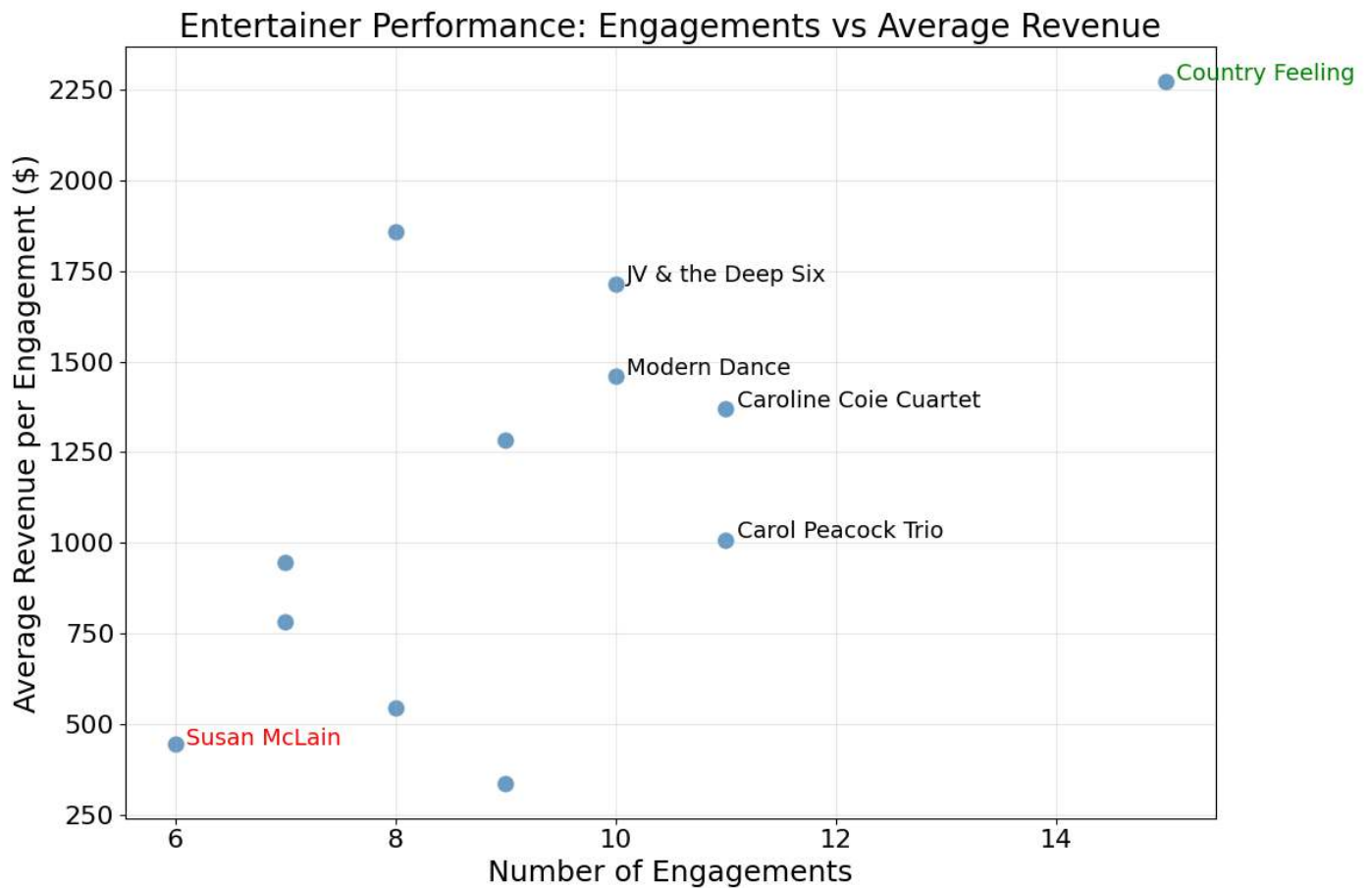
```

```

        fontsize=14,
        color="red"
    )

plt.title("Entertainer Performance: Engagements vs Average Revenue")
plt.xlabel("Number of Engagements")
plt.ylabel("Average Revenue per Engagement ($)")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



```

import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams.update({
    'font.size': 16,          # base font size
    'axes.titlesize': 20,     # title
    'axes.labelsize': 18,     # x/y labels
    'xtick.labelsize': 18,    # x ticks
    'ytick.labelsize': 18,    # y ticks
})

# Merge engagements with customers to get city info
df_market = dfs["engagements"].merge(dfs["customers"], on="customerid", how="left")

# Aggregate by city
market_summary = df_market.groupby("custcity").agg(
    total_revenue=("contractprice", "sum"),
    num_engagements=("engagementnumber", "count")
).reset_index()

# Compute average revenue per engagement for bubble size
market_summary["avg_revenue_per_eng"] = market_summary["total_revenue"] / market_summary["num_engagements"]

# Plot bubble chart
plt.figure(figsize=(12,7))
plt.scatter(
    market_summary["num_engagements"],
    market_summary["total_revenue"],
    s=market_summary["avg_revenue_per_eng"]*5, # adjust scale for visibility
    alpha=0.6,
    color="skyblue",
    edgecolor="black"
)

# Add labels for each city
for i, row in market_summary.iterrows():
    plt.text(
        row["num_engagements"] + 0.1,
        row["total_revenue"],
        row["custcity"],
        fontsize=9,
        verticalalignment="center"
    )

plt.xlabel("Number of Engagements")
plt.ylabel("Total Revenue ($)")
plt.title("Market Strength: Revenue vs Engagements by City")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

```


Market Strength: Revenue vs Engagements by City

